

UNIVERSIDAD
NACIONAL
DE COLOMBIA

Departamento de Ingeniería Mecánica y Mecatrónica

IA y Minirobots

Taller 5. Redes Neuronales Artificiales

Autores:

Yeison Esteban Ortega Chacua

Daniel Alberto Rodriguez Porras

Daniel Santiago Navarro Gil

Docente:

José Martínez

Bogotá D.C, Colombia

Desarrollo

1. Maximizar función

a. Planteamiento

El objetivo del ejercicio fue desarrollar dos redes neuronales artificiales capaces de aprender el comportamiento de las compuertas lógicas NAND y XOR. Para ello se siguió la estructura de aprendizaje supervisado comentado en clase, en la cual se proporcionan a la red tanto las entradas como las salidas esperadas, y posteriormente se ajustan los pesos y sesgos mediante retropropagación con el fin de reducir el error.

Las cuatro combinaciones posibles de entrada y los respectivos rótulos utilizados fueron:

x_1	x_2	NAND	XOR
0	0	1	0
0	1	1	1
1	0	1	1
1	1	0	0

Tabla 1. Combinaciones posibles de entradas para las compuertas lógicas a analizar.

A partir de ello, se entiende que los vectores de salida deseados serían $Y_{NAND} = [1, 1, 1, 0]$ y $Y_{XOR} = [0, 1, 1, 0]$.

Para ambas compuertas se utilizó una red con topología 2-3-3-1, compuesta por dos neuronas de entrada, dos capas ocultas de tres neuronas cada una y una única neurona de salida, dando lugar a la configuración requerida de dos capas ocultas.

b. Procedimiento

Ya como tal durante la ejecución de la red neuronal, el entrenamiento se realizó utilizando una red feedforward, donde la información se propaga desde la capa de entrada hasta la capa de salida, siguiendo la formulación presentada en la guía:

$$Z^{(i)} = B^{(i)} + (W^{(i)T})A^{(i-1)}$$

$$A^{(i)} = \sigma(Z^{(i)}),$$

donde W representa los pesos, B los sesgos, Z la combinación lineal de cada capa y A su respectiva salida, lo que busca generalizar la propagación entre las diferentes capas de una red neuronal.

Además, se aplica como función de activación la vista en clase (función sigmoide):

$$\sigma(z) = \frac{1}{1+e^{-z}}, \text{ la cual, derivando se expresa como } \sigma'(z) = \sigma(z)(1 - \sigma(z)).$$

Esto debido a que esta función permite introducir comportamiento no lineal en la red y su derivada es utilizada durante el proceso de retropropagación.

A partir de esta ejecución, se realizó en cada iteración primero la propagación hacia adelante a través de las tres capas de procesamiento de esta forma:

$$X \rightarrow A^{(1)} \rightarrow A^{(2)} \rightarrow A^{(3)}$$

donde la salida $A^{(3)}$ fue comparada con el rótulo esperado Y_r , calculando el error como la diferencia entre ambas ($e = Y_r - A^{(3)}$).

A partir del cálculo de este error se aplicó el algoritmo de backpropagation, iniciando desde la capa de salida y propagando la corrección hacia las capas anteriores, buscando así determinar cuánto debía modificarse cada peso y sesgo, llegando a tener como el error se calcula en la salida y posteriormente se transmite hacia atrás para actualizar los parámetros de todas las capas.

Respecto a la actualización de los datos se hizo mediante descenso de gradiente con una tasa de aprendizaje de $\eta = 0.5$ y se empezó realizando unas 50000 épocas de entrenamiento para cada red. Finalmente, con el valor de la función de pérdida se graficó su disminución conforme avanzaba el entrenamiento.

c. Resultados

Red para NAND

Considerando el código ejecutado en google colab [2] archivo Ejercicio_1_Redes_NAND_XOR se puede reconocer que antes del entrenamiento, la red NAND producía valores cercanos entre sí para las cuatro entradas [0.7607, 0.7626, 0.7632, 0.7649], por lo que inicialmente no era capaz de distinguir correctamente los patrones.

Sin embargo, cuando se aplican las épocas, se obtuvo:

$$\hat{Y}_{NAND} = [0.9985, 0.9934, 0.9933, 0.0114]$$

de la cual, si se toma un umbral de clasificación de 0.5, considerando 1 a todo $Y \geq 1$, y el resto 0, se obtiene [1, 1, 1, 0], lo que coincide exactamente con la tabla de verdad de la compuerta NAND.

```

... Salida continua NAND:
  [[0.9985 0.9934 0.9933 0.0114]]

Salida clasificada NAND:
  [[1 1 1 0]]

Salida esperada:
  [[1 1 1 0]]

```

Figura 1. Resultados obtenidos en la ejecución del algoritmo para compuerta NAND

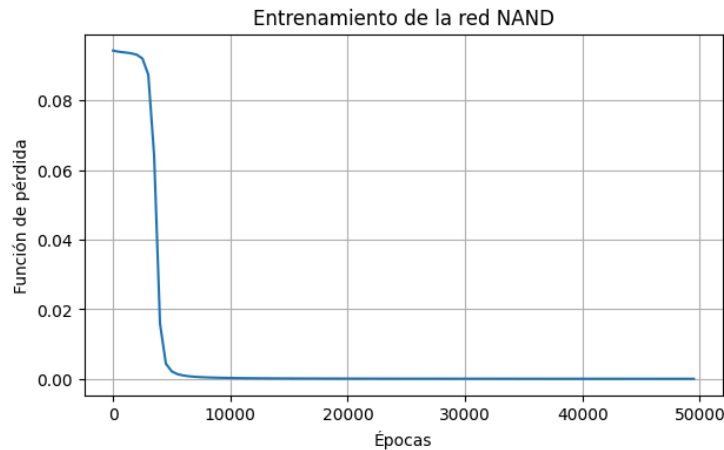


Figura 2. Gráfica de comportamiento del entrenamiento de la red, respecto a las épocas aplicadas para la compuerta NAND.

Red para XOR

Para XOR, la salida inicial de la red también era prácticamente igual para todas las entradas: [0.7924, 0.7947, 0.7937, 0.7958]. Por lo tanto, después del entrenamiento se obtuvo:

$$\hat{Y}_{XOR} = [0.0135, 0.9889, 0.9889, 0.0127]$$

E igualmente al aplicar el umbral de 0.5, la clasificación resultante fue [0, 1, 1, 0] que corresponde exactamente a la tabla de verdad de XOR

```

... Salida continua XOR:
  [[0.0135 0.9889 0.9889 0.0127]]

Salida clasificada XOR:
  [[0 1 1 0]]

Salida esperada:
  [[0 1 1 0]]

```

Figura 3. Resultados obtenidos en la ejecución del algoritmo para compuerta XOR.



Figura 4. Gráfica de comportamiento del entrenamiento de la red, respecto a las épocas aplicadas para la compuerta XOR.

Las dos redes lograron aprender correctamente sus respectivas funciones lógicas. En ambos casos, las salidas correspondientes a la clase 1 terminaron próximas a uno, mientras que las correspondientes a la clase 0 se aproximaron a cero.

Para el caso de XOR, sus datos no son linealmente separables, por lo cual, una única neurona no puede resolver correctamente este problema, así que el uso de capas ocultas y funciones de activación no lineales permite construir representaciones internas más complejas y así llegar a aprender la función XOR.

Por otro lado, NAND constituye un patrón más sencillo, que una red de menor tamaño podría resolver este caso, sin embargo se mantuvieron dos capas ocultas para cumplir con las condiciones establecidas en el ejercicio y utilizar una estructura uniforme.

2. Clasificación de prendas de vestir

a. Planteamiento

El ejercicio busca implementar una red neuronal artificial utilizando la librería TensorFlow para clasificar prendas de vestir pertenecientes al dataset Fashion MNIST. A diferencia del ejercicio anterior, en el cual se implementó manualmente el proceso de propagación y retropropagación, en este caso TensorFlow permite definir la arquitectura de la red y realizar automáticamente el entrenamiento.

b. Procedimiento

El problema corresponde a un caso de **aprendizaje supervisado**, puesto que cada imagen del conjunto de datos está asociada con un rótulo que indica la clase a la que pertenece. Es decir, en este se proporcionan tanto las características de entrada como las salidas esperadas, y la red ajusta sus pesos con el fin de reducir el error entre su predicción y el valor real.

Fashion MNIST contiene imágenes de prendas en escala de grises, con dimensiones de 28×28 píxeles, distribuidos en diez categorías. Las clases utilizadas fueron:

Etiqueta	Tabla
0	Camiseta
1	Pantalón
2	Suéter
3	Vestido
4	Abrigo
5	Sandalia
6	Camisa
7	Tenis
8	Bolso
9	Botín

Tabla 2. Clases a usar obtenidas de Fashion MNIST

Al cargar el dataset mediante TensorFlow se obtienen conjuntos separados de entrenamiento y prueba. Las imágenes inicialmente contenían intensidades de píxel entre 0 y 255, por lo que se realizó una normalización dividiendo todos los valores entre 255 y de esta forma, cada píxel queda representado por un valor entre 0 y 1. Esta transformación no modifica la información visual de la imagen, pero facilita el proceso numérico de entrenamiento al trabajar con datos en una escala uniforme. Así, cada imagen de 28×28 píxeles contiene en total:

$$28 \times 28 = 784$$

valores. Antes de introducirla en la red, la matriz bidimensional se transforma en un vector de 784 características, por tanto, para la red neuronal una prenda no se representa inicialmente como una “camiseta” o un “zapato”, sino como un conjunto de valores numéricos correspondientes a sus píxeles. El propósito del entrenamiento consiste en encontrar relaciones entre esos valores que permitan diferenciar las distintas categorías.

Arquitectura y lógica aplicada

Se implementó una red neuronal completamente conectada con la estructura:

$$784 \rightarrow 128 \rightarrow 10$$

Aquí, la primera etapa corresponde a los 784 valores obtenidos al convertir cada imagen en un vector, posteriormente se utiliza una capa oculta formada por 128 neuronas. Cada una de estas neuronas recibe información proveniente de los píxeles y realiza una combinación ponderada mediante pesos y sesgos. De esta manera, esta operación sigue el mismo modelo matemático presentado en clase:

$$z = b + \sum_j w_j x_j$$

sobre cuyo resultado se aplica posteriormente una función de activación, esto con el fin de introducir no linealidad y permitir que una red pueda representar relaciones más complejas que una simple combinación lineal.

En la capa oculta se utilizó la función ReLU, definida como $ReLU(z) = \max(0, z)$, la cual, mantiene los valores positivos y convierte en cero los negativos, permitiendo así introducir no linealidad y favorece un entrenamiento eficiente, especialmente cuando la red debe identificar patrones complejos.

La última capa posee diez neuronas, una para cada categoría de Fashion MNIST y sobre estas salidas se aplica la función Softmax, cuyo propósito es transformar los valores obtenidos en una distribución de probabilidades, es decir, si yo tuviera una imagen con una salida conceptual como [0.01, 0.00, 0.02, 0.03, 0.01, 0.04, 0.02, 0.12, 0.03, 0.72], la mayor probabilidad corresponde a la **posición 9**, por lo cual la red clasificaría la imagen como un botín, por ende, el modelo no entrega directamente un nombre sino diez probabilidades, y se selecciona como respuesta la categoría con el mayor valor.

De esta manera, la propagación completa puede resumirse como:

$$Imagen \rightarrow 784 \text{ px} \rightarrow 128 \text{ neuronas} \rightarrow 10 \text{ probabilidades} \rightarrow \text{clase predicha}$$

Entrenamiento

Para ello se utilizó como función de pérdida `sparse_categorical_crossentropy`, adecuada para un problema de clasificación en el que existen varias categorías y cada imagen posee un rótulo entero entre 0 y 9. Esta permite determinar qué tan diferente es la respuesta producida por la red respecto a la clase correcta. Si la red asigna una probabilidad alta a la categoría real, el error será reducido, en cambio, si otorga poca probabilidad a la respuesta correcta, la pérdida será mayor.

A partir de este error se ejecuta el algoritmo de retropropagación o backpropagation. La lógica es la misma estudiada en el ejercicio anterior, lo que sería primero se obtiene una predicción mediante propagación hacia adelante, posteriormente se calcula el error y este se transmite desde la capa de salida hacia las capas anteriores con el fin de determinar cuánto debe modificarse cada peso, solo que aquí cambia que no se implementa manualmente sino con TensorFlow se realiza automáticamente el cálculo de los gradientes y la retropropagación.

El entrenamiento se realizó durante diez épocas utilizando grupos de 32 imágenes, donde una época corresponde a un recorrido completo del conjunto de entrenamiento, mientras que trabajar con grupos o batches permite realizar varias actualizaciones de los pesos durante una misma época.

Además, se reservó el 20 % de los datos de entrenamiento como conjunto de validación. De esta forma fue posible comparar el comportamiento de la red sobre datos utilizados para aprender y sobre datos que no participan directamente en la actualización de los pesos, pudiendo analizar así la capacidad de generalización del modelo y detectar problemas como el subentrenamiento o el sobreentrenamiento.



Figura 5. Ejemplos de imágenes pertenecientes al conjunto de entrenamiento Fashion MNIST.

c. Resultados

Durante el entrenamiento se registraron tanto la exactitud como la función de pérdida, conforme a esto, se puede esperar que al avanza las épocas, la exactitud aumenta mientras la pérdida disminuye, lo que representa que las predicciones realizadas por la red se aproximan progresivamente a las categorías correctas.

Parámetro	Valor resultante
Exactitud de entrenamiento	91,12%

Exactitud final de validación	88,24%
Exactitud en prueba	87,23%
Pérdida en prueba	0,03551
Imágenes incorrectamente clasificadas	1277/10000

Tabla 3. Resultados de las métricas estimadas para la solución del Ejercicio 2.

Cómo la exactitud de entrenamiento, validación y prueba permanecen relativamente cercana, se puede concluir que la red logró aprender características generales de las prendas y no únicamente memorizar las imágenes utilizadas durante el entrenamiento.

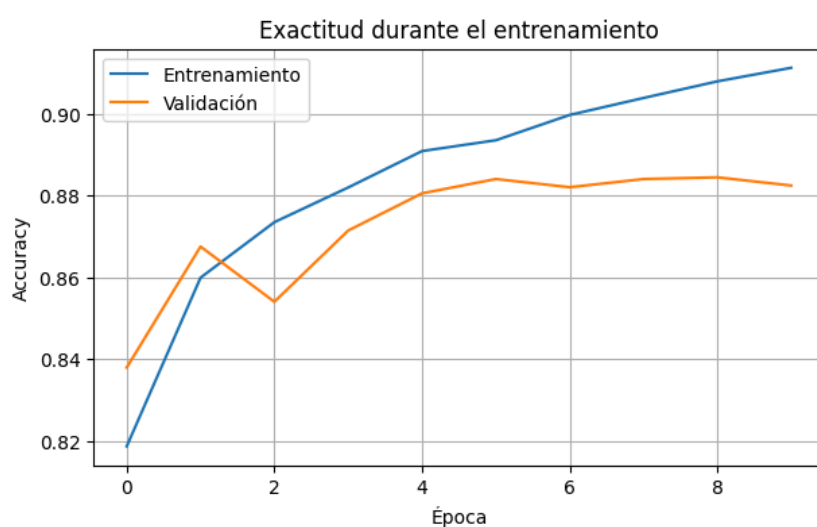


Figura 6. Evolución de la exactitud durante el entrenamiento y la validación del modelo.

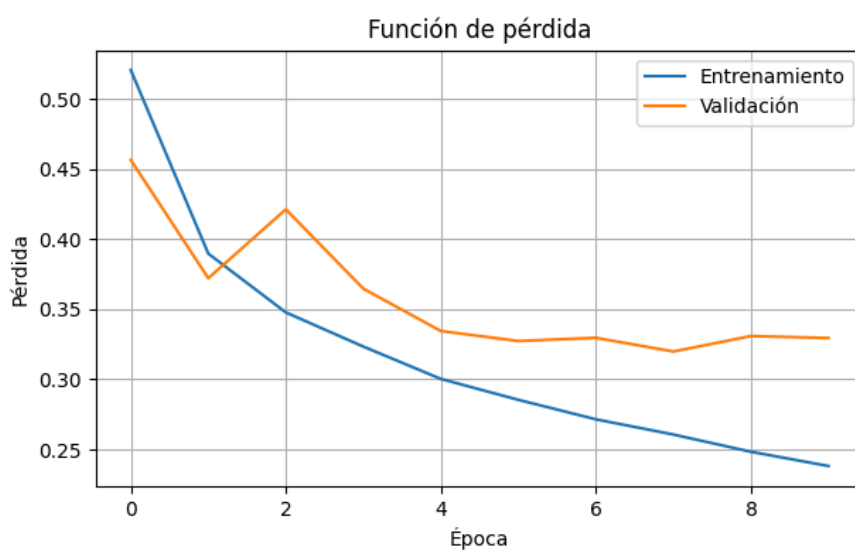


Figura 7. Evolución de la función de pérdida para los conjuntos de entrenamiento y validación.



Figura 8. Comparación entre las categorías reales (R) y las predicciones realizadas (P) por la red neuronal sobre imágenes del conjunto de prueba.

Adicionalmente, se construyó una matriz de confusión, en la cual cada fila representa la categoría real y cada columna la categoría predicha. Los valores ubicados sobre la diagonal principal representan clasificaciones correctas, mientras que los valores fuera de dicha diagonal muestran errores cometidos por el modelo..

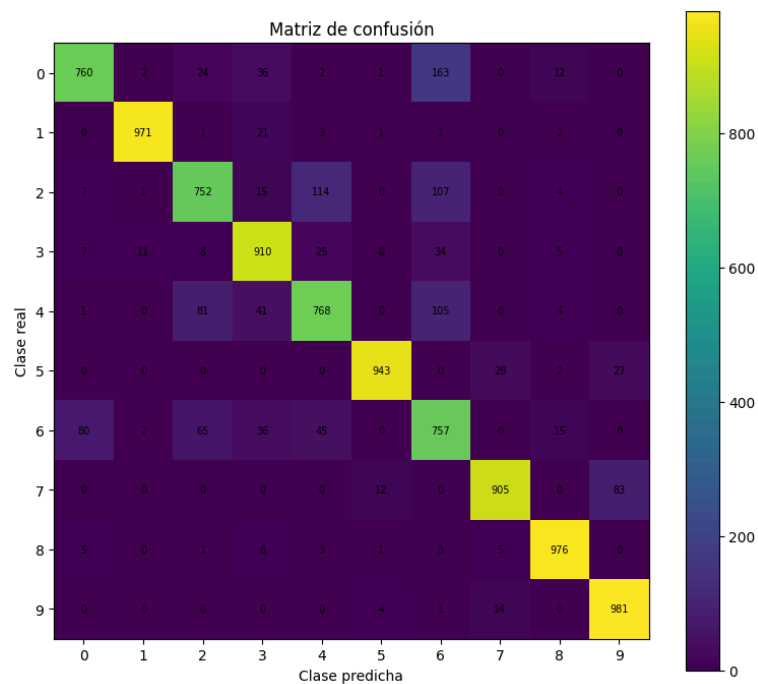


Figura 9. Matriz de confusión obtenida para la clasificación de las diez categorías de Fashion MNIST. Las filas representan las clases reales y las columnas las clases predichas.

A partir de esta, se puede observar que no todas las categorías presentan el mismo nivel de dificultad, ya que las mayores confusiones tienden a producirse entre prendas visualmente similares, como camisetas, camisas, suéteres y abrigos, debido a que las imágenes tienen baja resolución y algunas comparten formas semejantes. Por otro lado, prendas con características más diferenciadas, como bolsos o ciertos tipos de calzado, pueden resultar más fáciles de reconocer.

3. Librería de Kaggle para Python,

a. Planteamiento

Para desarrollar este punto se seleccionó el dataset **Breast Cancer Wisconsin (Diagnostic)** disponible públicamente en Kaggle. El objetivo del conjunto de datos es clasificar una muestra como correspondiente a un tumor benigno o maligno a partir de características obtenidas de imágenes digitalizadas de núcleos celulares. Kaggle describe directamente el problema como la predicción de si el tumor es benigno o maligno. [Kaggle](#)

La elección permite aplicar directamente los conceptos de aprendizaje supervisado presentados en el material del curso, en los cuales cada ejemplo se compone de un vector de características X y un rótulo Y . 5 Redes Neuronales1 2026 2

El dataset fue descargado directamente desde Python utilizando la librería **kagglehub**, de manera que no es necesario descargar manualmente el archivo desde el navegador. El rótulo utilizado corresponde a la variable **diagnosis**, que puede tomar los valores **M**, para maligno, y **B**, para benigno.

El conjunto contiene **30 características numéricas** derivadas de diez mediciones básicas realizadas sobre los núcleos celulares: radio, textura, perímetro, área, suavidad, compacidad, concavidad, puntos cóncavos, simetría y dimensión fractal. Para cada una se incluyen valores asociados a la media, al error estándar y al denominado peor valor, dando como resultado 30 variables predictoras. [Kaggle](#)

Antes del entrenamiento se eliminaron columnas que no representan información útil para el diagnóstico, como el identificador de la muestra, y se transformó el rótulo a una representación binaria:

$$M = 1, B = 0$$

Posteriormente los datos se dividieron en un conjunto de entrenamiento correspondiente al 80 % de las observaciones y un conjunto de prueba correspondiente al 20 %. Esta metodología es coherente con el material del curso, que plantea la necesidad de separar los datos para poder estimar la capacidad del modelo de generalizar a observaciones no utilizadas durante el ajuste. 5

Redes Neuronales1 2026 2

Las variables de entrada fueron estandarizadas antes de entrenar la red neuronal. Este paso es necesario debido a que las características presentan escalas diferentes; por ejemplo, el área puede tomar valores considerablemente mayores que la suavidad o la dimensión fractal. La estandarización evita que una variable influya excesivamente sobre el entrenamiento únicamente por su escala numérica.

Se implementó una red neuronal artificial utilizando TensorFlow y Keras. La arquitectura empleada está formada por una capa de entrada de 30 características, una primera capa oculta de 32 neuronas con activación ReLU, una segunda capa oculta de 16 neuronas con activación ReLU y una neurona de salida con activación sigmoideal.

La utilización de una función sigmoideal en la salida permite interpretar el resultado como una probabilidad entre cero y uno. Esta idea está directamente relacionada con las funciones de activación estudiadas en el documento del curso, donde se explica que la función de activación introduce la no linealidad necesaria para que la red neuronal pueda modelar relaciones complejas. 5

Redes Neuronales1 2026 2

Debido a que se trata de una clasificación binaria, la función de pérdida seleccionada fue `binary_crossentropy`, mientras que el optimizador utilizado fue Adam. Durante el entrenamiento también se empleó `EarlyStopping`, encargado de detener el proceso cuando la pérdida de validación deja de mejorar y recuperar posteriormente los pesos correspondientes al mejor resultado.

Para evaluar el modelo se utilizaron las métricas accuracy, precision, recall y área bajo la curva ROC, además de una matriz de confusión. El análisis de la matriz de confusión es particularmente importante en este problema porque permite identificar falsos negativos, es decir, casos realmente malignos que fueron clasificados por el modelo como benignos.

b. Resultados

Reporte de clasificación:

	precision	recall	f1-score	support
Benigno	0.96	1.00	0.98	72
Maligno	1.00	0.93	0.96	42
accuracy		0.97	114	
macro avg	0.98	0.96	0.97	114
weighted avg	0.97	0.97	0.97	114

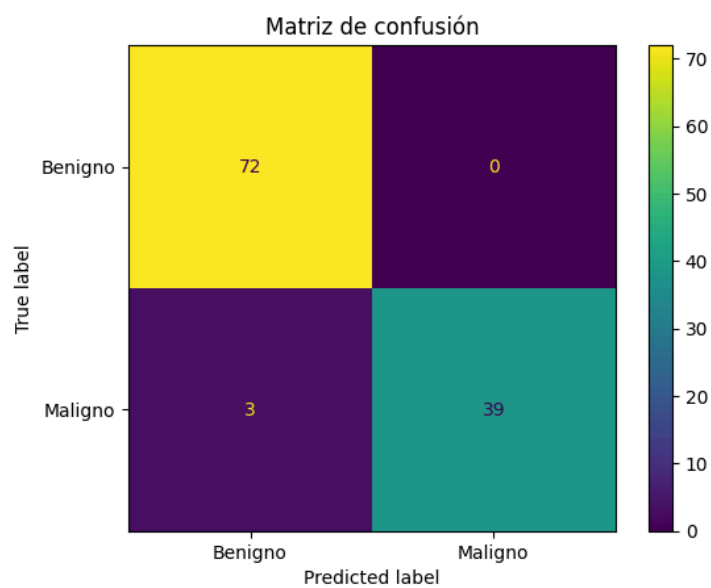


Figura 10. Matriz de confusión del modelo de clasificación.

El modelo clasificó correctamente las 72 muestras benignas y 39 de las 42 muestras malignas. Se presentaron tres falsos negativos, correspondientes a muestras malignas clasificadas como benignas.

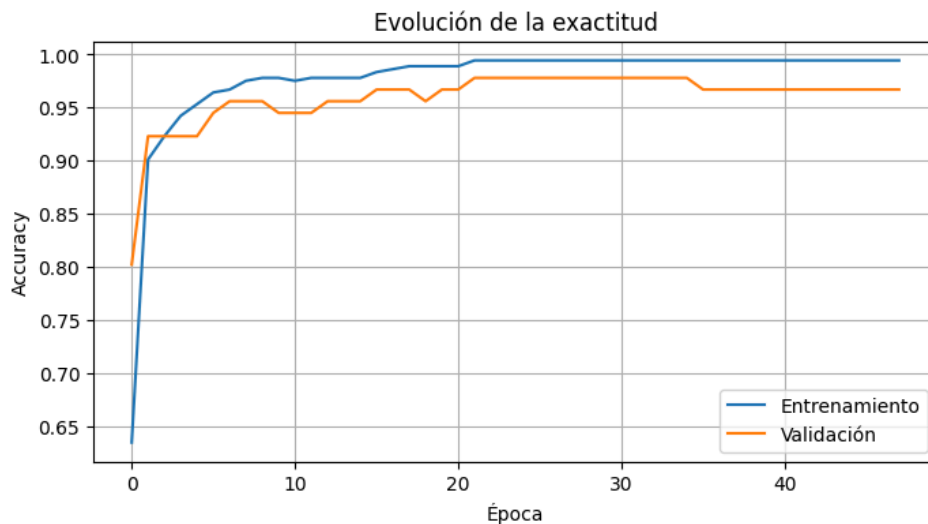
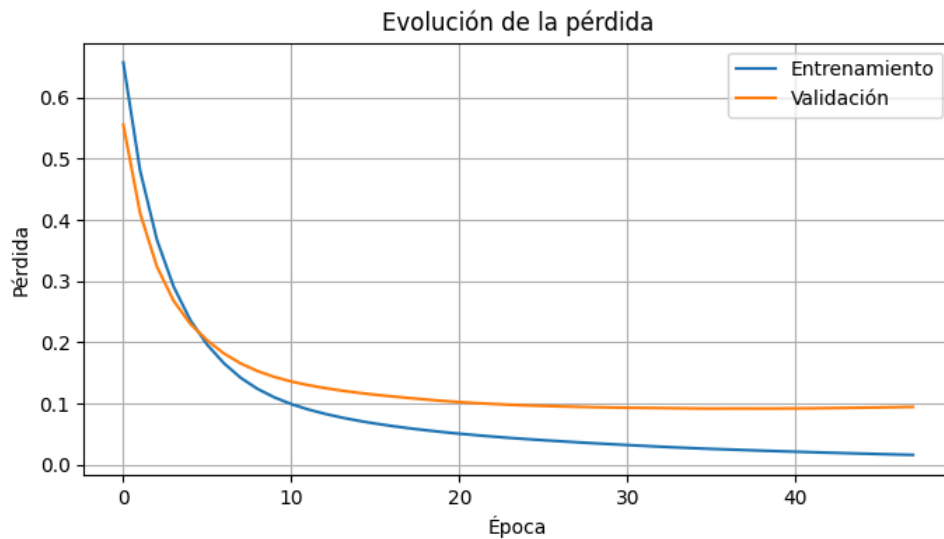


Figura 11 y 12. Evolución de la pérdida y la exactitud durante el entrenamiento.

Se observa una convergencia rápida durante las primeras épocas y una posterior estabilización de las métricas de validación. La diferencia entre entrenamiento y validación hacia las últimas épocas sugiere un leve sobreentrenamiento, aunque el desempeño general del modelo se mantiene alto.

El modelo de clasificación obtuvo un desempeño general alto sobre el conjunto de prueba. La exactitud alcanzada fue aproximadamente del **97 %**, lo que indica que la mayoría de las muestras fueron clasificadas correctamente como benignas o malignas. Para la clase **benigna** se

obtuvo una precisión de **0.96**, un recall de **1.00** y un F1-score de **0.98**. En el caso de la clase **maligna**, la precisión fue de **1.00**, el recall de **0.93** y el F1-score de **0.96**.

La matriz de confusión permite observar estos resultados con mayor detalle. De las **72 muestras benignas**, todas fueron clasificadas correctamente, por lo que no se presentaron casos benignos identificados erróneamente como malignos. Para las muestras malignas, **39 de 42** fueron reconocidas correctamente, mientras que **3 casos malignos fueron clasificados como benignos**. Este último resultado corresponde a los falsos negativos del modelo y constituye el error más importante en este tipo de aplicación, debido a que implica no detectar correctamente una muestra perteneciente a la clase maligna.

En cuanto al proceso de entrenamiento, la función de pérdida disminuyó de forma rápida durante las primeras épocas y posteriormente se estabilizó. La pérdida de entrenamiento continuó disminuyendo hasta valores muy bajos, mientras que la pérdida de validación se estabilizó alrededor de un valor cercano a **0.09**. Este comportamiento indica que la red aprendió correctamente los patrones principales del conjunto de datos. Sin embargo, la separación progresiva entre ambas curvas hacia las últimas épocas sugiere un **leve sobreentrenamiento**, ya que el modelo continúa mejorando sobre los datos de entrenamiento mientras que el desempeño sobre validación prácticamente deja de mejorar.

La evolución de la exactitud presenta un comportamiento similar. La precisión del conjunto de entrenamiento alcanza valores cercanos al **99 %**, mientras que la validación se mantiene aproximadamente entre **97 % y 98 %** después de las primeras 20 épocas. Esta estabilidad indica que el modelo logra una buena capacidad de generalización y que continuar aumentando el número de épocas no produce mejoras significativas sobre los datos no utilizados directamente para ajustar los pesos.

En conjunto, los resultados muestran que la arquitectura implementada es capaz de clasificar adecuadamente las muestras del dataset utilizado. El modelo obtiene un buen equilibrio entre precisión y recall para ambas clases, aunque los tres falsos negativos encontrados muestran que todavía existe margen de mejora. Para reducir este tipo de error podrían evaluarse cambios en la arquitectura de la red, el número de épocas, la tasa de aprendizaje o el umbral de decisión utilizado para clasificar una muestra como maligna.

4. Punto libre.

a. Planteamiento

Para el punto libre propuse desarrollar un problema de **regresión mediante redes neuronales**, de forma que complemente los problemas de clasificación realizados en los demás puntos del taller. El material distingue precisamente las dos aplicaciones principales de las redes neuronales: clasificación y predicción de valores numéricos. 5 Redes Neuronales1 2026 2

El problema seleccionado consiste en predecir el valor medio de viviendas utilizando el dataset **California Housing**. El conjunto posee ocho características de entrada relacionadas con las propiedades y su ubicación:

MedInc, ingreso medio; **HouseAge**, antigüedad promedio; **AveRooms**, número promedio de habitaciones; **AveBedrms**, número promedio de dormitorios; **Population**, población; **AveOccup**, ocupación promedio; **Latitude**; y **Longitude**.

La salida del modelo es una variable continua correspondiente al valor medio de la vivienda, expresado por el dataset en unidades de cien mil dólares. Debido a que la salida no pertenece a categorías discretas, este problema corresponde a una regresión y no a una clasificación.

Los datos se dividieron nuevamente en entrenamiento y prueba, y las características fueron normalizadas mediante **StandardScaler**. Posteriormente se diseñó una red neuronal formada por capas densas de 64, 32 y 16 neuronas, todas con función de activación ReLU. La última capa contiene únicamente una neurona lineal encargada de proporcionar el valor numérico estimado.

La función de pérdida utilizada fue el error cuadrático medio:

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

El modelo se evaluó mediante tres métricas complementarias. El error absoluto medio se definió como:

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

El RMSE corresponde a la raíz cuadrada del error cuadrático medio:

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

Finalmente, se calculó el coeficiente de determinación R^2 , utilizado para estimar qué proporción de la variabilidad del valor de las viviendas logra representar el modelo.

Para evaluar visualmente el entrenamiento se generó una gráfica de la pérdida para los conjuntos de entrenamiento y validación. También se construyó una gráfica de valores reales frente a valores predichos. En una predicción ideal, los puntos deberían localizarse sobre la diagonal $y = \hat{y}$; por tanto, mientras más cercanos se encuentren a dicha línea, menor es el error cometido por la red.

b. Resultados

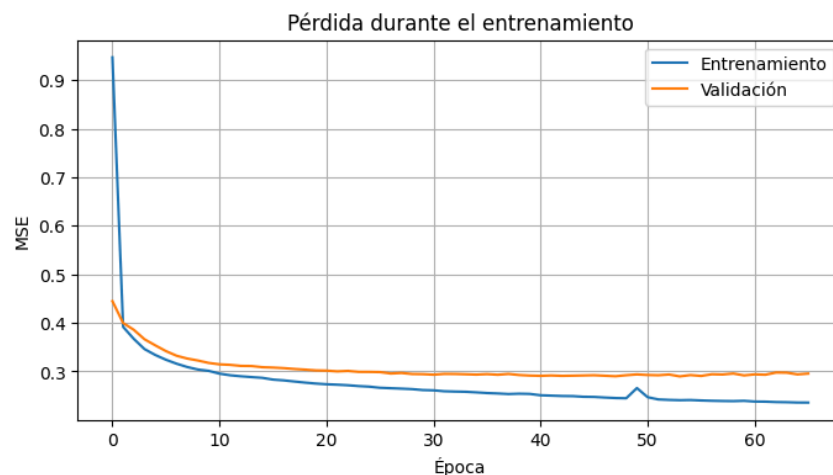


Figura 13. Evolución de la pérdida durante el entrenamiento

La pérdida disminuye rápidamente durante las primeras épocas y posteriormente se estabiliza. La cercanía entre las curvas de entrenamiento y validación indica una adecuada capacidad de generalización, sin evidencia de sobreentrenamiento severo.

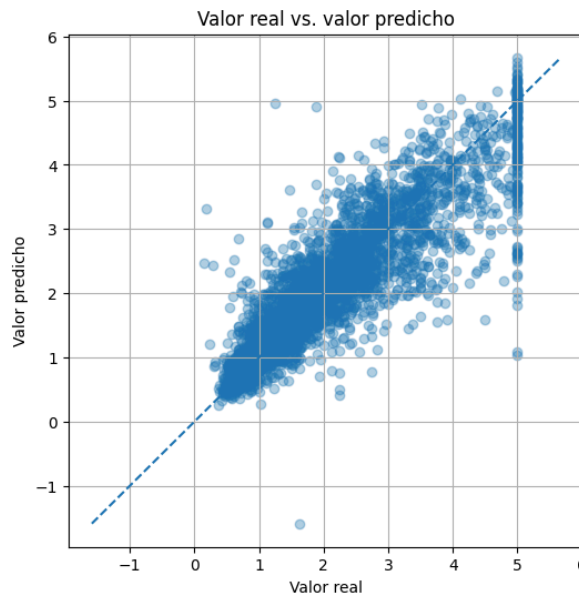


Figura 14. Comparación entre valores reales y valores predichos.

La mayoría de las predicciones siguen la tendencia de la diagonal ideal, aunque se presenta dispersión, especialmente para los valores más altos. El comportamiento obtenido es coherente con un modelo de regresión con un R^2 de 0.7762.

El modelo de regresión obtuvo un **MAE de 0.3684**, un **RMSE de 0.5415** y un coeficiente de determinación $R^2 = 0.7762$. Estos valores indican que el modelo logra representar adecuadamente la relación entre las características de entrada y el valor de las viviendas, aunque todavía existe una parte de la variabilidad que no logra explicar completamente.

El valor de $R^2 = 0.7762$ significa que aproximadamente el **77.62 % de la variabilidad del valor de las viviendas puede ser explicada por el modelo**. Para un problema de regresión con datos reales, este resultado muestra un desempeño aceptable y evidencia que la red neuronal aprendió patrones relevantes a partir de las características disponibles.

El **MAE de 0.3684** representa el error absoluto medio entre los valores reales y los valores predichos. Como la variable objetivo del dataset está expresada en unidades de **100 000 dólares**, este valor equivale aproximadamente a un error medio de **36 840 USD** por vivienda. Por su parte, el **RMSE de 0.5415** equivale aproximadamente a **54 150 USD**, siendo mayor que el MAE porque penaliza con mayor fuerza las predicciones que presentan errores grandes.

La gráfica de pérdida muestra que durante las primeras épocas se produce una disminución rápida del error tanto para entrenamiento como para validación. Posteriormente, ambas curvas

se estabilizan, con una pérdida de validación cercana a 0.30 y una pérdida de entrenamiento ligeramente inferior. La separación entre ambas curvas es relativamente pequeña, por lo que no se observa un sobreentrenamiento severo. El comportamiento general indica que la red alcanzó una zona de convergencia y que continuar aumentando significativamente el número de épocas probablemente produciría mejoras limitadas.

En la gráfica de **valor real frente a valor predicho** se observa una tendencia aproximadamente alineada con la diagonal ideal, lo cual confirma que el modelo reproduce correctamente la tendencia general de los datos. Sin embargo, existe una dispersión visible alrededor de la línea, especialmente para valores elevados de vivienda. También se observa una acumulación de muestras alrededor del valor real de 5, asociada al límite superior presente en el propio dataset. Esto dificulta que el modelo reproduzca con total precisión los valores más altos.

En conjunto, los resultados muestran que la red neuronal desarrollada tiene una capacidad adecuada para realizar predicciones continuas a partir de las características del dataset. Aunque no alcanza una predicción perfecta, logra explicar cerca del 78 % de la variabilidad presente en los datos y mantiene errores relativamente moderados. El ejercicio demuestra la aplicación de redes neuronales a un problema de **regresión**, complementando los ejercicios de clasificación desarrollados previamente.

Referencias

[1] J. J. Martínez P., Redes Neuronales, material de clase, sept. 2026.

[2] Rodríguez, D., Ortega Y., Navarro, S. (2026). IA-y-Minirobots [Repositorio de código]. [GitHub](#).

Nota: En la elaboración de este documento se utilizó ChatGPT como herramienta de apoyo para la organización, redacción y revisión del contenido. Las fuentes citadas fueron consultadas y verificadas por los autores